

Memoria del Proyecto GPS - Grupo GP10A

Miguel Gabaldón Poncela
Gonzalo García Martínez-Echevarría

1. Descripción de la estructura general del programa

La estructura consta de tres módulos principales entre los que se completan las tareas de procesamiento, cálculo y visualización. El programa está estructurado en tres módulos principales:

1. **gps.py:**

El programa `gps.py` está diseñado para calcular rutas óptimas dentro de una red de calles en Madrid, utilizando información de OpenStreetMap (OSM). El módulo principal, `gps.py`, actúa como el núcleo del programa, coordinando las funciones de los módulos auxiliares `callejero.py` y `grafo_pesado.py`.

Además, el programa principal en `gps.py` gestiona la interacción con el usuario, solicitando direcciones de origen y destino, así como el modo de cálculo de ruta deseado. También coordina la llamada a funciones para procesar estas direcciones, calcular rutas y generar instrucciones de navegación. Finalmente, utiliza herramientas gráficas para mostrar la red de calles y la ruta calculada.

Al ejecutar el `gps.py` por primera vez se mostrará un mapa de la ciudad antes de pedir las direcciones y luego después de introducir las direcciones de origen y destino.

2. **callejero.py:**

El módulo `callejero.py` se encarga de gestionar el archivo de direcciones proporcionado (`direcciones.csv`) y convertirlo en un formato estructurado para su uso en el programa. Este módulo también proporciona utilidades para procesar coordenadas, buscar direcciones y cargar un grafo de calles desde OSM. El módulo utiliza un `DataFrame` de `pandas` llamado `callejero` para almacenar la información del callejero, incluyendo el nombre, número, latitud y longitud de cada dirección.

3. **grafo_pesado.py:**

Por otro lado, `grafo_pesado.py` implementa algoritmos para el análisis de grafos pesados. Esto incluye el algoritmo de Dijkstra para encontrar caminos mínimos, así como los algoritmos de Prim y Kruskal para calcular árboles abarcadores mínimos. Estos algoritmos utilizan estructuras como colas de prioridad (`heapq`) para optimizar los cálculos, y están diseñados para ser reutilizables en otros contextos.

2. Resolución del problema de la recuperación de la ruta óptima y de las direcciones de navegación

El cálculo de rutas se basa en un grafo dirigido (`nx.MultiDiGraph`) que representa la red de calles. Los nodos corresponden a intersecciones o puntos específicos, mientras que las aristas contienen atributos como longitud (`length`), tipo de calle (`highway`) y velocidad máxima (`maxspeed`). Dependiendo del modo seleccionado por el usuario, el programa elige una función de coste que determina cómo se calcula el peso de cada arista.

Para la ruta más corta, se utiliza la longitud de las calles como función de coste, lo que minimiza la distancia recorrida. En el caso de la ruta más rápida, el coste se basa en el tiempo estimado, calculado como la longitud dividida por la velocidad máxima. Si no se dispone de información de velocidad, se asume un valor por defecto de 50 km/h. La ruta optimizada añade una

Memoria del Proyecto GPS - Grupo GP10A

Miguel Gabaldón Poncela
Gonzalo García Martínez-Echevarría

penalización basada en la probabilidad de detenerse en semáforos, considerando un tiempo adicional de 30 segundos con un 80% de probabilidad.

La recuperación de la ruta se realiza mediante el algoritmo de Dijkstra, adaptado para considerar la función de coste seleccionada. Una vez calculado el camino, se generan instrucciones para guiar al usuario. Estas instrucciones describen cada segmento de la ruta, incluyendo la distancia a recorrer, el nombre de la calle, el tipo de vía y la velocidad máxima permitida. Además, se indica si es necesario girar a la izquierda o a la derecha en los cruces.

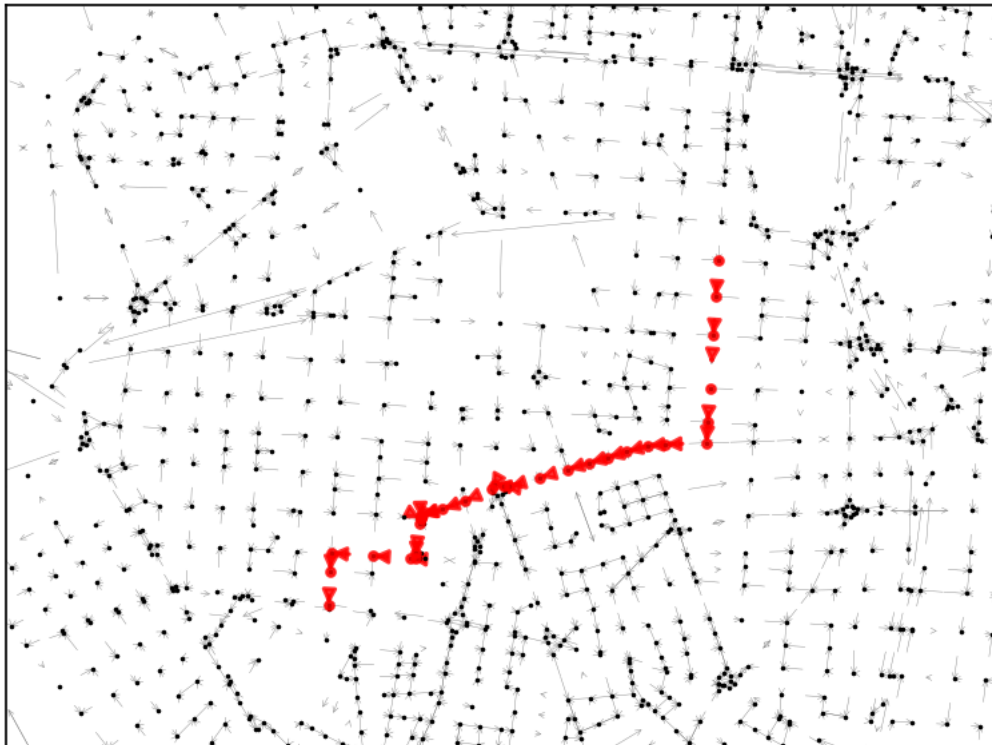


Figura 1: Ejemplo de la ruta calculada

3. Elección de funciones de coste y generación de instrucciones

El programa admite tres modos de cálculo de ruta:

1. Más corta:

- Coste basado en la longitud de las calles ('length').
- Función de peso: $\text{peso} = \lambda G, u, v: G[u][v].\text{get}('length', 1)$
- Calcula rutas que minimizan la distancia recorrida.

2. Más rápida:

- Coste basado en el tiempo estimado, considerando la velocidad máxima de las calles ('maxspeed').
- Función de peso: $\text{peso} = \lambda G, u, v: G[u][v].\text{get}('length', 1) / G[u][v].\text{get}('maxspeed', 50)$

Memoria del Proyecto GPS - Grupo GP10A

Miguel Gabaldón Poncela
Gonzalo García Martínez-Echevarría

- Se usa un valor por defecto de 50 km/h si no se dispone de información de velocidad.

3. Optimizada (semaforización):

- Incluye penalizaciones por posibles semáforos en las intersecciones.
- Función de peso: $\text{peso} = \lambda G, u, v: (G[u][v].get('length', 1) / G[u][v].get('maxspeed', 50)) + 30 * 0.8$
- Penalización basada en una probabilidad de 80% de detenerse.

Instrucciones proporcionadas al usuario

- Para cada segmento de la ruta, se calcula:
 - La distancia a recorrer.
 - El nombre de la calle y el tipo de vía (highway).
 - La velocidad máxima permitida.
 - Posibles giros (derecha o izquierda) en los cruces o que termine la calle y empiece una nueva.
- Por ejemplo, una instrucción típica podría ser: " Después de avanzar 961 metros, gira a la derecha en Calle de Cea Bermúdez."

4. Comentarios sobre el diseño

1. Eficiencia:

- Nos aseguramos que los cálculos de caminos mínimos (Dijkstra) sean eficientes con el uso de la librería heapq.
- Los algoritmos de Prim y Kruskal se optimizan al manejar los árboles abarcadores mínimos.

2. Escalabilidad:

- Si se crearan nuevas funciones de coste se podrían añadir fácilmente al programa
- Como usamos la librería OSM podemos aplicar el mismo programa con otras ciudades diferentes.

3. Modularidad:

- La separación en tres módulos permite reutilizar código (por ejemplo, la carga y visualización de grafos).

5. Bibliografía

Nos hemos ayudado de Chatgpt para entender algunos conceptos y para ayudarnos con librerías en el código que no sabíamos implementar